In [1]:
```
# Import libraries.
from pathlib import Path
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

sns.set_theme(style="whitegrid")
pd.set_option("display.max_columns", 40)
DATA = Path("data")
```

In [2]:
```
# Read the five CSV files into separate DataFrames.
customers = pd.read_csv("C:/Users/hp/Documents/Marketing_Campaign_Analysis/data/customers.csv")
transactions = pd.read_csv("C:/Users/hp/Documents/Marketing_Campaign_Analysis/data/transactions.csv")
products = pd.read_csv("C:/Users/hp/Documents/Marketing_Campaign_Analysis/data/products.csv")
events = pd.read_csv("C:/Users/hp/Documents/Marketing_Campaign_Analysis/data/events.csv")
campaigns = pd.read_csv("C:/Users/hp/Documents/Marketing_Campaign_Analysis/data/campaigns.csv")
```

In [3]:
```
# Keep the names and their DataFrames together in one dictionary.
for name, df in{"Customers": customers, "Transactions": transactions, "Events": events,
                "Products": products, "Campaigns": campaigns}.items():
# Repeat the same inspection for every DataFrame.
    print(f"{name}: rows={len(df):,}, columns={df.shape[1]}")
```

```
Customers: rows=100,000, columns=7
Transactions: rows=103,127, columns=9
Events: rows=2,000,000, columns=12
Products: rows=2,000, columns=6
Campaigns: rows=50, columns=7
```

In [4]:
```
# Displaying each table's first five rows.
for name, df in {"Customers": customers, "Transactions": transactions, "Events": events,
                 "Products": products, "Campaigns": campaigns}.items():
    print(f"{name}: \n{df.head()}\n")
```

```
Customers:
   customer_id signup_date country  age  gender loyalty_tier  \
0            1  2021-04-08      BR   48    Male       Bronze
1            2  2023-04-28      IN   36  Female       Silver
2            3  2022-12-18      UK   35  Female       Silver
3            4  2022-04-26      US   45    Male       Silver
4            5  2022-04-20      IN   53    Male       Silver

  acquisition_channel
0            Referral
1             Organic
2             Organic
3         Paid Search
4             Organic

Transactions:
   transaction_id            timestamp  customer_id  product_id  quantity  \
0               1  2021-12-27 08:25:15        59540      1630.0         3
1               2  2023-06-06 21:14:26        54871      1901.0         3
2               3  2023-08-31 05:29:54        51818      1884.0         1
3               4  2022-06-26 20:33:46        18164      1114.0         2
4               5  2023-07-26 18:12:35        86915       408.0         1

   discount_applied  gross_revenue  campaign_id  refund_flag
0              0.00          43.74            0            0
1              0.00         174.78           21            0
2              0.00          40.61           37            0
3              0.15          68.76           13            0
4              0.00          14.64            4            0

Events:
   event_id            timestamp  customer_id  session_id   event_type  \
0         1  2021-01-14 13:35:43        43812      535101         view
1         2  2021-12-03 21:36:50        71340       96426  add_to_cart
2         3  2021-12-27 08:25:15        59540      220126     purchase
3         4  2022-01-22 15:06:54         3601      484555  add_to_cart
4         5  2021-05-10 12:03:09        92735       60646       bounce

   product_id device_type traffic_source  campaign_id page_category  \
0      1004.0     desktop          Email           43           PLP
1       986.0     desktop          Email           10           PDP
2      1630.0      mobile        Organic            0           PDP
```

```
3      1532.0     desktop    Paid Search           30      Checkout
4         NaN     desktop          Email           26           PLP

   session_duration_sec experiment_group
0                 115.1          Control
1                  32.4        Variant_A
2                 190.7        Variant_A
3                 134.8        Variant_B
4                  53.1        Variant_A

Products:
   product_id     category     brand  base_price launch_date  is_premium
0           1      Grocery  Brand_58       14.19  2021-08-02           0
1           2      Fashion   Brand_1       25.80  2021-09-14           0
2           3  Electronics  Brand_70      165.46  2021-01-18           1
3           4      Fashion  Brand_56       75.45  2023-03-03           1
4           5       Sports   Brand_1       72.50  2022-04-19           1

Campaigns:
   campaign_id      channel     objective  start_date    end_date  \
0            1  Paid Search    Cross-sell  2021-10-25  2021-11-26
1            2        Email     Retention  2021-10-24  2021-12-24
2            3        Email  Reactivation  2023-10-08  2023-11-30
3            4      Display  Reactivation  2022-07-25  2022-10-07
4            5       Social   Acquisition  2022-07-09  2022-09-29

  target_segment  expected_uplift
0   Deal Seekers            0.022
1   Deal Seekers            0.116
2     Churn Risk            0.100
3   Deal Seekers            0.111
4  New Customers            0.144
```

In [7]:
```
for name, df in{"Customers": customers, "Transactions": transactions, "Events": events,
                "Products": products, "Campaigns": campaigns}.items():
    print(f"{name}: \n{df.info()}\n")
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 100000 entries, 0 to 99999
Data columns (total 7 columns):
 #   Column               Non-Null Count   Dtype 
---  ------               --------------   ----- 
 0   customer_id          100000 non-null  int64 
 1   signup_date          100000 non-null  object
 2   country              100000 non-null  object
 3   age                  100000 non-null  int64 
 4   gender               100000 non-null  object
 5   loyalty_tier         100000 non-null  object
 6   acquisition_channel  100000 non-null  object
dtypes: int64(2), object(5)
memory usage: 5.3+ MB
Customers:
None

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 103127 entries, 0 to 103126
Data columns (total 9 columns):
 #   Column            Non-Null Count   Dtype  
---  ------            --------------   -----  
 0   transaction_id    103127 non-null  int64  
 1   timestamp         103127 non-null  object 
 2   customer_id       103127 non-null  int64  
 3   product_id        92678 non-null   float64
 4   quantity          103127 non-null  int64  
 5   discount_applied  103127 non-null  float64
 6   gross_revenue     92678 non-null   float64
 7   campaign_id       103127 non-null  int64  
 8   refund_flag       103127 non-null  int64  
dtypes: float64(3), int64(5), object(1)
memory usage: 7.1+ MB
Transactions:
None

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 2000000 entries, 0 to 1999999
Data columns (total 12 columns):
 #   Column                Dtype  
---  ------                -----  
 0   event_id              int64
```

```
 1   timestamp             object
 2   customer_id           int64
 3   session_id            int64
 4   event_type            object
 5   product_id            float64
 6   device_type           object
 7   traffic_source        object
 8   campaign_id           int64
 9   page_category         object
 10  session_duration_sec  float64
 11  experiment_group      object
dtypes: float64(2), int64(4), object(6)
memory usage: 183.1+ MB
Events:
None

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 2000 entries, 0 to 1999
Data columns (total 6 columns):
 #   Column       Non-Null Count  Dtype
---  ------       --------------  -----
 0   product_id   2000 non-null   int64
 1   category     2000 non-null   object
 2   brand        2000 non-null   object
 3   base_price   2000 non-null   float64
 4   launch_date  2000 non-null   object
 5   is_premium   2000 non-null   int64
dtypes: float64(1), int64(2), object(3)
memory usage: 93.9+ KB
Products:
None

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 50 entries, 0 to 49
Data columns (total 7 columns):
 #   Column           Non-Null Count  Dtype
---  ------           --------------  -----
 0   campaign_id      50 non-null     int64
 1   channel          50 non-null     object
 2   objective        50 non-null     object
 3   start_date       50 non-null     object
 4   end_date         50 non-null     object
```

```
 5   target_segment   50 non-null     object
 6   expected_uplift  50 non-null     float64
dtypes: float64(1), int64(1), object(5)
memory usage: 2.9+ KB
Campaigns:
None
```

In [8]:
```
for name, df in {"Customers": customers, "Transaction": transactions, "Events": events,
                 "Products": products, "Campaigns": campaigns}.items():
    print(f"{name}: \n{df.describe(include="all")}\n")
```

```
Customers:
         customer_id signup_date country            age  gender loyalty_tier  \
count   100000.000000      100000  100000  100000.000000  100000       100000
unique            NaN        1095       7            NaN       3            4
top               NaN  2022-11-04      US            NaN    Male       Bronze
freq              NaN         124   34931            NaN   48054        60276
mean     50000.500000         NaN     NaN      35.224370     NaN          NaN
std      28867.657797         NaN     NaN       9.595792     NaN          NaN
min          1.000000         NaN     NaN      18.000000     NaN          NaN
25%      25000.750000         NaN     NaN      28.000000     NaN          NaN
50%      50000.500000         NaN     NaN      35.000000     NaN          NaN
75%      75000.250000         NaN     NaN      42.000000     NaN          NaN
max     100000.000000         NaN     NaN      70.000000     NaN          NaN

       acquisition_channel
count               100000
unique                   5
top                Organic
freq                 30200
mean                   NaN
std                    NaN
min                    NaN
25%                    NaN
50%                    NaN
75%                    NaN
max                    NaN

Transaction:
        transaction_id            timestamp    customer_id   product_id  \
count    103127.000000               103127  103127.000000  92678.00000
unique             NaN               103074            NaN          NaN
top                NaN  2022-08-01 05:14:31            NaN          NaN
freq               NaN                    2            NaN          NaN
mean      51564.000000                  NaN   49946.662707    999.63139
std       29770.344943                  NaN   28841.644812    576.19187
min           1.000000                  NaN       2.000000      1.00000
25%       25782.500000                  NaN   25081.000000    498.00000
50%       51564.000000                  NaN   49957.000000   1002.00000
75%       77345.500000                  NaN   74870.500000   1498.00000
max      103127.000000                  NaN  100000.000000   2000.00000

            quantity  discount_applied  gross_revenue    campaign_id  \
```

```
count   103127.000000     103127.000000   92678.000000  103127.000000
unique            NaN               NaN           NaN           NaN
top               NaN               NaN           NaN           NaN
freq              NaN               NaN           NaN           NaN
mean         1.381539          0.041379     90.355493     19.945252
std          0.748176          0.060286    100.684683     16.379672
min          1.000000          0.000000   -783.040000      0.000000
25%          1.000000          0.000000     34.240000      4.000000
50%          1.000000          0.000000     68.000000     18.000000
75%          1.000000          0.050000    117.617500     34.000000
max          4.000000          0.200000   1858.320000     50.000000

          refund_flag
count   103127.000000
unique            NaN
top               NaN
freq              NaN
mean         0.029372
std          0.168846
min          0.000000
25%          0.000000
50%          0.000000
75%          0.000000
max          1.000000

Events:
            event_id            timestamp   customer_id    session_id  \
count   2.000000e+06              2000000  2.000000e+06  2.000000e+06
unique           NaN              1977065           NaN           NaN
top              NaN  2023-09-18 07:00:22           NaN           NaN
freq             NaN                    4           NaN           NaN
mean    1.000000e+06                  NaN  5.001209e+04  3.330514e+05
std     5.773504e+05                  NaN  2.887960e+04  1.924715e+05
min     1.000000e+00                  NaN  1.000000e+00  1.000000e+00
25%     5.000008e+05                  NaN  2.501000e+04  1.662308e+05
50%     1.000000e+06                  NaN  5.000900e+04  3.328190e+05
75%     1.500000e+06                  NaN  7.503800e+04  4.997712e+05
max     2.000000e+06                  NaN  1.000000e+05  6.666660e+05

       event_type    product_id device_type traffic_source   campaign_id  \
count     2000000  1.799629e+06     1959700        2000000  2.000000e+06
unique          5           NaN           3             10           NaN
```

```
top          view           NaN      mobile        Organic           NaN
freq      1043573           NaN     1176146         776758           NaN
mean          NaN  1.000100e+03         NaN            NaN  1.276203e+01
std           NaN  5.776716e+02         NaN            NaN  1.634052e+01
min           NaN  1.000000e+00         NaN            NaN  0.000000e+00
25%           NaN  5.000000e+02         NaN            NaN  0.000000e+00
50%           NaN  1.000000e+03         NaN            NaN  0.000000e+00
75%           NaN  1.501000e+03         NaN            NaN  2.600000e+01
max           NaN  2.000000e+03         NaN            NaN  5.000000e+01

       page_category  session_duration_sec experiment_group
count        2000000          2.000000e+06          2000000
unique             5                   NaN                3
top              PLP                   NaN          Control
freq          601284                   NaN          1198404
mean             NaN          1.306426e+02              NaN
std              NaN          1.590376e+02              NaN
min              NaN          1.000000e-01              NaN
25%              NaN          5.810000e+01              NaN
50%              NaN          1.017000e+02              NaN
75%              NaN          1.640000e+02              NaN
max              NaN          7.533800e+03              NaN

Products:
        product_id     category    brand   base_price launch_date  \
count   2000.000000         2000     2000  2000.000000        2000
unique          NaN            6      100          NaN         901
top             NaN  Electronics  Brand_7          NaN  2023-10-23
freq            NaN          455       32          NaN           7
mean    1000.500000          NaN      NaN    72.166650         NaN
std      577.494589          NaN      NaN    52.315792         NaN
min        1.000000          NaN      NaN     5.110000         NaN
25%      500.750000          NaN      NaN    32.505000         NaN
50%     1000.500000          NaN      NaN    61.245000         NaN
75%     1500.250000          NaN      NaN    97.955000         NaN
max     2000.000000          NaN      NaN   464.580000         NaN

        is_premium
count   2000.000000
unique          NaN
top             NaN
freq            NaN
```

```
mean       0.500000
std        0.500125
min        0.000000
25%        0.000000
50%        0.500000
75%        1.000000
max        1.000000

Campaigns:
        campaign_id      channel     objective  start_date    end_date  \
count      50.00000           50            50          50          50
unique          NaN            5             4          49          48
top             NaN  Paid Search  Reactivation  2022-07-25  2022-09-29
freq            NaN           11            15           2           2
mean       25.50000          NaN           NaN         NaN         NaN
std        14.57738          NaN           NaN         NaN         NaN
min         1.00000          NaN           NaN         NaN         NaN
25%        13.25000          NaN           NaN         NaN         NaN
50%        25.50000          NaN           NaN         NaN         NaN
75%        37.75000          NaN           NaN         NaN         NaN
max        50.00000          NaN           NaN         NaN         NaN

       target_segment  expected_uplift
count              50        50.000000
unique              5              NaN
top     New Customers              NaN
freq               12              NaN
mean              NaN         0.086660
std               NaN         0.035854
min               NaN         0.022000
25%               NaN         0.057250
50%               NaN         0.092500
75%               NaN         0.113750
max               NaN         0.144000
```

In [9]:
```
# checking missing values
for name, df in{"Customers": customers, "Transactions": transactions, "Events": events,
                "Products": products, "Campaigns": campaigns}.items():
    audit = pd.DataFrame({"missing_values":df.isna().sum(), "missing_rate":(df.isna().mean()*100).round(2)})
    print("\n", name.upper())
```

```
    print(audit)
    print("\n Duplicated values:", "\n", df.duplicated().sum())
```

```
 CUSTOMERS
                    missing_values  missing_rate
customer_id                     0           0.0
signup_date                     0           0.0
country                         0           0.0
age                             0           0.0
gender                          0           0.0
loyalty_tier                    0           0.0
acquisition_channel             0           0.0

 Duplicated values:
 0

 TRANSACTIONS
                  missing_values  missing_rate
transaction_id                 0          0.00
timestamp                      0          0.00
customer_id                    0          0.00
product_id                 10449         10.13
quantity                       0          0.00
discount_applied               0          0.00
gross_revenue              10449         10.13
campaign_id                    0          0.00
refund_flag                    0          0.00

 Duplicated values:
 0

 EVENTS
                     missing_values  missing_rate
event_id                         0          0.00
timestamp                        0          0.00
customer_id                      0          0.00
session_id                       0          0.00
event_type                       0          0.00
product_id                  200371         10.02
device_type                  40300          2.02
traffic_source                   0          0.00
campaign_id                      0          0.00
page_category                    0          0.00
session_duration_sec             0          0.00
experiment_group                 0          0.00
```

```
Duplicated values:
0

PRODUCTS
             missing_values  missing_rate
product_id                0           0.0
category                  0           0.0
brand                     0           0.0
base_price                0           0.0
launch_date               0           0.0
is_premium                0           0.0

Duplicated values:
0

CAMPAIGNS
                 missing_values  missing_rate
campaign_id                   0           0.0
channel                       0           0.0
objective                     0           0.0
start_date                    0           0.0
end_date                      0           0.0
target_segment                0           0.0
expected_uplift               0           0.0

Duplicated values:
0
```

In [5]:
```
# Filling missing device type
events["device_type"]= events["device_type"].fillna("unknown")
# Compare event types with whether the product ID is missing.
product_missing_by_event = pd.crosstab(
    events["event_type"],
    events["product_id"].isna())
product_missing_by_event.columns = ["Product_ID_Available", "Product_ID_Missing"]
product_missing_by_event
```

Out[5]:

event_type	Product_ID_Available	Product_ID_Missing
add_to_cart	284370	0
bounce	0	189922
click	379008	0
purchase	92678	10449
view	1043573	0

In [6]:
```
# Convert the listed date columns to proper datetime values.
for df, col in [(customers, "signup_date"), (products, "launch_date"),
                (campaigns, "start_date"), (campaigns, "end_date"),
                (transactions, "timestamp"), (events, "timestamp")]:
    df[col] = pd.to_datetime(df[col], errors="coerce")

# Store product IDs as nullable integers; missing IDs stay missing.
for df in [products, transactions, events]:
    df["product_id"] = df["product_id"].astype("Int64")

# Remove spaces and use consistent text types.
for col in ["country","gender","loyalty_tier","acquisition_channel"]:
    customers[col] = customers[col].astype("string").str.strip()
for col in ["event_type","device_type","traffic_source","page_category","experiment_group"]:
    events[col] = events[col].astype("string").str.strip()
events["event_type"] = events["event_type"].str.lower()
events["traffic_source"] = events["traffic_source"].str.title()
events["device_type"] = events["device_type"].str.lower()
# Translate country codes into readable country names.
customers["country"] = customers["country"].replace(
    {"US":"United States","UK":"United Kingdom","IN":"India","BR":"Brazil","CA":"Canada","DE":"Germany","AU":"Austra

# Campaign ID 0 means no campaign; other values indicate an attributed ID.
for df in [transactions, events]:
    df["campaign_status"] = np.where(df["campaign_id"].eq(0), "No Campaign", "Campaign Attributed")

print("Cleaned event types:", events["event_type"].value_counts().to_dict())
```

```
Cleaned event types: {'view': 1043573, 'click': 379008, 'add_to_cart': 284370, 'bounce': 189922, 'purchase': 103127}
```

In [7]:
```
# Add human-readable labels for premium and non-premium products.
products["Premium_status"] = np.where(
    products["is_premium"] ==1,
    "Premium",
    "Not_premium")
```

In [8]:
```
# Check whether non-missing product IDs exist in the product table.
print("Unknown transaction products:", (~transactions["product_id"].dropna().isin(products["product_id"])).sum())
print("Unknown event products:", (~events["product_id"].dropna().isin(products["product_id"])).sum())
# Check whether customer IDs exist in the customer table.
print("Unknown transaction customers:", (~transactions["customer_id"].isin(customers["customer_id"])).sum())
print("Unknown event customers:", (~events["customer_id"].isin(customers["customer_id"])).sum())
# Count rows that have campaign ID zero (no campaign attribution).
print("No-campaign transaction rows:", transactions["campaign_id"].eq(0).sum())
print("No-campaign event rows:", events["campaign_id"].eq(0).sum())

# Join signup dates only for this time-quality check.
timeline_tx = transactions[["customer_id","timestamp"]].merge(customers[["customer_id","signup_date"]], on="customer_
                                                              validate="many_to_one")
timeline_ev = events[["customer_id","timestamp"]].merge(customers[["customer_id","signup_date"]], on="customer_id",
                                                        validate="many_to_one")
# Count records that happened before the recorded signup date.
print("Transactions before signup:", (timeline_tx["timestamp"] < timeline_tx["signup_date"]).sum())
print("Events before signup:", (timeline_ev["timestamp"] < timeline_ev["signup_date"]).sum())
del timeline_tx, timeline_ev
# Compare website traffic source with campaign attribution status.
Events_campaign_attributed_unattributed_traffic_source = pd.crosstab(
    events["traffic_source"],
    events["campaign_id"] == 0)
Events_campaign_attributed_unattributed_traffic_source.columns =["Has_campaign", "Has_no_campaign"]
Events_campaign_attributed_unattributed_traffic_source
```

```
Unknown transaction products: 0
Unknown event products: 0
Unknown transaction customers: 0
Unknown event customers: 0
No-campaign transaction rows: 20955
No-campaign event rows: 1000251
Transactions before signup: 50593
Events before signup: 998530
```

Out[8]:

traffic_source	Has_campaign	Has_no_campaign
Direct	0	199762
Email	299640	0
Organic	0	800489
Paid Search	399838	0
Social	300271	0

In [9]:
```
# Merge product information to each transaction.
sales_analysis = transactions.merge(products[["product_id","category","brand","base_price","is_premium", "Premium_sta
                                    on="product_id", how="left", validate="many_to_one")
# Calculate revenue before discount.
sales_analysis["revenue_before_discount"] = sales_analysis["quantity"] * sales_analysis["base_price"]
# Calculate discount amount.
sales_analysis["discount_amount"] = sales_analysis["revenue_before_discount"] * sales_analysis["discount_applied"]
sales_analysis["signed_revenue"] = np.where(sales_analysis["refund_flag"].eq(1), -1, 1) * (
    sales_analysis["revenue_before_discount"] - sales_analysis["discount_amount"])
# Compare only rows with both a recorded and calculated revenue amount.
complete = sales_analysis["gross_revenue"].notna() & sales_analysis["signed_revenue"].notna()
print("Complete revenue rows:", complete.sum())
print("Matches validated formula (%):", round(np.isclose(sales_analysis.loc[complete,"gross_revenue"],sales_analysis.
                                                         complete,"signed_revenue"],atol=.01).mean()*100,2))
# Count rows whose product ID and recorded revenue are both missing.
print("Rows missing both product and revenue:", (sales_analysis["product_id"].isna()
                                                 & sales_analysis["gross_revenue"].isna()).sum())
# Count rows whose product ID and recorded revenue are both missing.
print("Refund rows with missing revenue:", (sales_analysis["refund_flag"].eq(1)
                                            & sales_analysis["gross_revenue"].isna()).sum())

# Use the refund flag to give refunded units a negative sign.
sales_analysis["signed_units_by_refund_flag"] = np.where(sales_analysis["refund_flag"].eq(1),
                                                         -sales_analysis["quantity"], sales_analysis["quantity"])
# Use the sign of known revenue for an alternative units measure.
sales_analysis["known_revenue_signed_units"] = sales_analysis["quantity"] * np.sign(sales_analysis["gross_revenue"])
```

```
sales_analysis["known_revenue_signed_units"] = (
    pd.to_numeric(sales_analysis["known_revenue_signed_units"], errors="coerce").round().astype("Int64"))
```

```
Complete revenue rows: 92678
Matches validated formula (%): 100.0
Rows missing both product and revenue: 10449
Refund rows with missing revenue: 325
```

In [10]:
```
# create customers age group column
customers["age_group"] = pd.cut(customers["age"], bins=[17, 24, 34, 44, 54, 64, np.inf],
                                labels=["18-24", "25-34", "35-44", "45-54", "55-64", "65+"])
# Extract year, month, quarter and weekday from dates.
for df in [sales_analysis, events]:
    df["year"] = df["timestamp"].dt.year
    df["year_month"] = df["timestamp"].dt.to_period("M")
    df["quarter"] = "Q" + df["timestamp"].dt.quarter.astype("string")
    df["days_of_week"] = df["timestamp"].dt.day_name()
# Add one so both the start date and end date count as campaign days.
campaigns["campaign_duration_days"] = (campaigns["end_date"] - campaigns["start_date"]).dt.days + 1
print(campaigns["campaign_duration_days"].describe())
```

```
count    50.000000
mean     51.980000
std      22.824521
min       9.000000
25%      38.250000
50%      55.000000
75%      69.000000
max      90.000000
Name: campaign_duration_days, dtype: float64
```

In [11]:
```
# Filter completed sales
completed_sales = sales_analysis.loc[sales_analysis["refund_flag"].eq(0) & sales_analysis["gross_revenue"].notna()].c

# Put every KPI name and its calculation into a dictionary.
kpi_dict = {
    "Total Transactions": sales_analysis["transaction_id"].nunique(),
    "Registered Customers": customers["customer_id"].nunique(),
    "Customers With Transactions": sales_analysis["customer_id"].nunique(),
    "Non-Refunded Units": sales_analysis.loc[sales_analysis["refund_flag"].eq(0), "quantity"].sum(),
    "Net Units From Refund Flags": sales_analysis["signed_units_by_refund_flag"].sum(),
    "Net Units With Known Revenue": sales_analysis["known_revenue_signed_units"].sum(),
    "Recorded Net Revenue": sales_analysis["gross_revenue"].sum(),
```

```
    "Refunded Transactions": sales_analysis["refund_flag"].sum(),
    "Known Refund Amount": -sales_analysis.loc[sales_analysis["gross_revenue"].lt(0), "gross_revenue"].sum(),
    "Average Completed Transaction Value": completed_sales["gross_revenue"].mean(),
    "Original Discount Amount, Known Products": sales_analysis.loc[sales_analysis["refund_flag"].eq(0), "discount_amo
    .sum(),
    # Add the refund rate right into the dictionary
    "Refund Rate (%)": round(sales_analysis["refund_flag"].mean() * 100, 2)
}
# Convert the dictionary into a two-column table.
kpi_table = pd.DataFrame(list(kpi_dict.items()), columns=["Metric", "Value"])

# Format results for display without changing the underlying calculations.
kpi_table["Value"] = kpi_table["Value"].apply(lambda x: f"{x:,.2f}" if x % 1 != 0 else f"{int(x):,}")

# Display the table cleanly
print(kpi_table.to_string(index=False))
```

```
                                Metric        Value
                  Total Transactions      103,127
                 Registered Customers      100,000
            Customers With Transactions       64,035
                     Non-Refunded Units      138,263
            Net Units From Refund Flags      134,052
          Net Units With Known Revenue      120,528
                 Recorded Net Revenue 8,373,966.36
                Refunded Transactions        3,029
                  Known Refund Amount   256,302.95
   Average Completed Transaction Value        95.92
Original Discount Amount, Known Products   373,183.94
                        Refund Rate (%)         2.94
```

In [12]:
```
# Merge customer information to each sales.
customer_analysis = sales_analysis.merge(customers[["customer_id","country","age_group","gender",
                                        "loyalty_tier","acquisition_channel"]],
                            on="customer_id", how="left", validate="many_to_one")
# Which loyalty tiers contribute the most recorded revenue?
loyalty_tier_performance =(
    customer_analysis.groupby("loyalty_tier")
    .agg(
        Customers=("customer_id", "nunique"),
        Transactions=("transaction_id", "nunique"),
        Refunds=("refund_flag","sum"),
```

```
        Signed_units_sold=("known_revenue_signed_units", "sum"),
        Recorded_revenue=("gross_revenue", "sum"))
    .reset_index()
    .sort_values("Recorded_revenue", ascending=False))

loyalty_tier_performance["Revenue_per_customer"] =(round(
    loyalty_tier_performance["Recorded_revenue"] / loyalty_tier_performance["Customers"],2))

loyalty_tier_performance["Average_Transaction_Value"] = (round(
    loyalty_tier_performance["Recorded_revenue"] / loyalty_tier_performance["Transactions"],2))

loyalty_tier_performance
```

Out[12]:

	loyalty_tier	Customers	Transactions	Refunds	Signed_units_sold	Recorded_revenue	Revenue_per_customer	Average_Transac
0	Bronze	36246	55232	1631	64653	4477838.77	123.54	
3	Silver	16855	28005	816	32713	2302086.76	136.58	
1	Gold	8705	15906	458	18605	1275144.30	146.48	
2	Platinum	2229	3984	124	4557	318896.53	143.07	

In [13]:

```
# Which countries generate the most customers, transactions and revenue?
country_performance = (
    customer_analysis.groupby("country")
    .agg(
        Customers=("customer_id", "nunique"),
        Transactions=("transaction_id", "nunique"),
        Refunds=("refund_flag","sum"),
        Signed_units_sold=("known_revenue_signed_units", "sum"),
        Recorded_revenue=("gross_revenue", "sum"))
    .reset_index()
    .sort_values("Recorded_revenue", ascending=False))

country_performance["Revenue_per_Customer"] = (round(country_performance["Recorded_revenue"] /
                                                     country_performance["Customers"],2))
country_performance["Average_Transaction_Value"] = (round(country_performance["Recorded_revenue"] /
                                                          country_performance["Transactions"],2))
country_performance
```

Out[13]:

	country	Customers	Transactions	Refunds	Signed_units_sold	Recorded_revenue	Revenue_per_Customer	Average_Transactio
6	United States	22296	35981	1010	42174	2950087.57	132.31	
4	India	12838	20727	598	24274	1679708.93	130.84	
5	United Kingdom	6507	10311	315	12050	844255.29	129.75	
1	Brazil	6453	10392	293	12132	836797.43	129.68	
2	Canada	6356	10193	322	11894	817855.39	128.67	
3	Germany	5070	8173	259	9495	654466.43	129.09	
0	Australia	4515	7350	232	8509	590795.32	130.85	

In [14]:

```
# How does purchasing behavior differ across age groups?
age_performance = (
    customer_analysis.groupby("age_group", observed=True)
    .agg(
        Customers=("customer_id", "nunique"),
        Transactions=("transaction_id", "nunique"),
        Refunds=("refund_flag","sum"),
        Signed_units_sold=("known_revenue_signed_units", "sum"),
        Recorded_revenue=("gross_revenue", "sum"))
    .reset_index()
    .sort_values("Recorded_revenue", ascending=False))

age_performance["revenue_per_customer"] =(round(age_performance["Recorded_revenue"] / age_performance["Customers"],2)

age_performance["Average_Transaction_Value"] =(round(age_performance["Recorded_revenue"] / age_performance["Transacti

age_performance
```

Out[14]:

	age_group	Customers	Transactions	Refunds	Signed_units_sold	Recorded_revenue	revenue_per_customer	Average_Transacti
2	35-44	22659	36309	1059	42518	2966010.54	130.90	
1	25-34	21279	34438	998	40106	2786236.67	130.94	
0	18-24	9227	14855	413	17357	1209729.34	131.11	
3	45-54	9216	14850	485	17391	1199351.17	130.14	
4	55-64	1538	2484	69	2934	197719.07	128.56	
5	65+	116	191	5	222	14919.57	128.62	

In [15]:

```
# How do recorded transactions and revenue differ by gender?
gender_performance = (
    customer_analysis.groupby("gender")
    .agg(
        Customers=("customer_id", "nunique"),
        Transactions=("transaction_id", "nunique"),
        Refunds=("refund_flag","sum"),
        Signed_units_sold=("known_revenue_signed_units", "sum"),
        Recorded_revenue=("gross_revenue", "sum"))
    .reset_index()
    .sort_values("Recorded_revenue", ascending=False))

gender_performance["revenue_per_customer"] =(round(gender_performance["Recorded_revenue"] / gender_performance["Custo

gender_performance["Average_Transaction_Value"] =(round(
    gender_performance["Recorded_revenue"] / gender_performance["Transactions"],2))

gender_performance
```

Out[15]:

	gender	Customers	Transactions	Refunds	Signed_units_sold	Recorded_revenue	revenue_per_customer	Average_Transaction_
0	Female	30785	49506	1466	57785	4022613.19	130.67	
1	Male	30732	49489	1437	57876	4008965.79	130.45	
2	Other	2518	4132	126	4867	342387.38	135.98	

In [16]:

```
# Which customer acquisition channels bring customers who later generate the strongest purchase activity?
acquisition_channel_performance = (
    customer_analysis.groupby("acquisition_channel")
    .agg(
        Customers=("customer_id", "nunique"),
        Transactions=("transaction_id", "nunique"),
        Refunds=("refund_flag","sum"),
        Signed_units_sold=("known_revenue_signed_units", "sum"),
        Recorded_revenue=("gross_revenue", "sum"))
    .reset_index()
    .sort_values("Recorded_revenue", ascending=False))
acquisition_channel_performance["revenue_per_customer"] =(round(
    acquisition_channel_performance["Recorded_revenue"] / acquisition_channel_performance["Customers"],2))

acquisition_channel_performance
```

Out[16]:

	acquisition_channel	Customers	Transactions	Refunds	Signed_units_sold	Recorded_revenue	revenue_per_customer
1	Organic	19370	31148	890	36611	2565334.77	132.44
2	Paid Search	19251	31112	925	36124	2499671.93	129.85
0	Email	9499	15331	420	18086	1255819.95	132.21
4	Social	9585	15297	497	17624	1218334.54	127.11
3	Referral	6330	10239	297	12083	834805.17	131.88

In [17]:

```
# Which categories generate the most revenue and have the highest percentage of refunded transactions?
category_performance = (
    sales_analysis.groupby("category")
    .agg(
```

```
        Customers=("customer_id", "nunique"),
        Transactions=("transaction_id", "nunique"),
        Refunds=("refund_flag","sum"),
        Signed_units_sold=("known_revenue_signed_units", "sum"),
        Recorded_revenue=("gross_revenue", "sum"))
    .reset_index()
    .sort_values("Recorded_revenue", ascending=False))
category_performance["Average_Transaction_value"] =(round(
    category_performance["Recorded_revenue"] / category_performance["Transactions"],2))
category_performance["Refund_rate"] =(round(
    category_performance["Refunds"] / category_performance["Transactions"]*100, 2))
category_performance
```

Out[17]:

	category	Customers	Transactions	Refunds	Signed_units_sold	Recorded_revenue	Average_Transaction_value	Refund_rate
1	Electronics	18897	21095	626	27500	3452007.19	163.64	2.97
4	Home	16793	18421	507	23860	1996432.94	108.38	2.75
2	Fashion	17560	19339	540	25291	1298075.77	67.12	2.79
5	Sports	9652	10168	329	13219	965609.13	94.97	3.24
0	Beauty	8801	9224	286	11918	369671.88	40.08	3.10
3	Grocery	13390	14431	416	18740	292169.45	20.25	2.88

In [18]:

```
# Which brands have the highest recorded revenue?
brand_ranking = (
    sales_analysis
    .groupby("brand")["gross_revenue"]
    .sum()
    .reset_index(name="Revenue"))
brand_ranking["Dense_Rank"] = (brand_ranking["Revenue"]
    .rank(method="dense", ascending=False).astype(int))
top_brands = brand_ranking[brand_ranking["Dense_Rank"] <= 5].sort_values("Dense_Rank")

top_brands
```

Out[18]:

	brand	Revenue	Dense_Rank
68	Brand_70	146317.73	1
70	Brand_72	131260.39	2
67	Brand_7	130946.29	3
84	Brand_85	126313.05	4
17	Brand_24	122461.33	5

In [19]:
```
# How does recorded revenue differ between premium and non-premium products?
premium_performance =(
    sales_analysis.groupby("Premium_status")
    .agg(
        Customers=("customer_id", "nunique"),
        Transactions=("transaction_id", "nunique"),
        Refunds=("refund_flag","sum"),
        Signed_units_sold=("known_revenue_signed_units", "sum"),
        Recorded_revenue=("gross_revenue", "sum"))
    .reset_index()
    .sort_values("Recorded_revenue", ascending=False))
premium_performance["Average_transaction_value"] = (round(
    premium_performance["Recorded_revenue"]/premium_performance["Transactions"],2))
premium_performance
```

Out[19]:

	Premium_status	Customers	Transactions	Refunds	Signed_units_sold	Recorded_revenue	Average_transaction_value
1	Premium	37039	46511	1385	60371	6451614.45	138.71
0	Not_premium	36889	46167	1319	60157	1922351.91	41.64

In [21]:
```
# What share of recorded transactions and revenue is linked to a campaign ID?
campaign_analysis = sales_analysis.merge(campaigns, on="campaign_id", how="left", validate="many_to_one")
campaign_status_performance = (
    campaign_analysis
    .groupby("campaign_status")
    .agg(
        Customers=("customer_id", "nunique"),
        Transactions=("transaction_id", "nunique"),
```

```
        Refunds=("refund_flag","sum"),
        Signed_units_sold=("known_revenue_signed_units", "sum"),
        Recorded_revenue=("gross_revenue", "sum"))
    .reset_index()
    .sort_values("Recorded_revenue", ascending=False))
campaign_status_performance["Transaction_Share_%"] = (round(campaign_status_performance["Transactions"] /
    campaign_analysis["transaction_id"].nunique()* 100,2))
campaign_status_performance["Revenue_Share_%"] = (round(campaign_status_performance["Recorded_revenue"] /
    campaign_status_performance["Recorded_revenue"].sum() * 100, 2))
campaign_status_performance
```

Out[21]:

	campaign_status	Customers	Transactions	Refunds	Signed_units_sold	Recorded_revenue	Transaction_Share_%	Revenue_Shar
0	Campaign Attributed	56079	82172	2415	96059	6683846.32	79.68	7
1	No Campaign	18689	20955	614	24469	1690120.04	20.32	2

In [22]:

```
# Which recorded campaign channels have the highest associated revenue?
channel_performance = (
    campaign_analysis
    .groupby("channel")
    .agg(
        Customers=("customer_id", "nunique"),
        Campaigns=("campaign_id", "nunique"),
        Transactions=("transaction_id", "nunique"),
        Refunds=("refund_flag","sum"),
        Signed_units_sold=("known_revenue_signed_units", "sum"),
        Recorded_revenue=("gross_revenue", "sum"))
    .reset_index()
    .sort_values("Recorded_revenue", ascending=False))
channel_performance["Average_transaction_value"] =(round(
    channel_performance["Recorded_revenue"] / channel_performance["Transactions"],2))
channel_performance
```

Out[22]:

	channel	Customers	Campaigns	Transactions	Refunds	Signed_units_sold	Recorded_revenue	Average_transaction_value
0	Affiliate	17855	11	19694	563	23200	1608144.17	81.66
3	Paid Search	17162	11	18841	581	21929	1533909.64	81.41
2	Email	15463	11	16741	493	19643	1357550.66	81.09
1	Display	13909	9	14995	422	17399	1211416.38	80.79
4	Social	11180	8	11901	356	13888	972825.47	81.74

In [23]:

```
# How much recorded revenue is associated with campaigns whose objective is acquisition, retention, cross-sell or rec
objective_performance = (
    campaign_analysis
    .groupby("objective")
    .agg(
        Customers=("customer_id", "nunique"),
        Campaigns=("campaign_id", "nunique"),
        Transactions=("transaction_id", "nunique"),
        Refunds=("refund_flag","sum"),
        Signed_units_sold=("known_revenue_signed_units", "sum"),
        Recorded_revenue=("gross_revenue", "sum"))
    .reset_index()
    .sort_values("Recorded_revenue", ascending=False))
objective_performance["Average_transaction_value"] =(round(
    objective_performance["Recorded_revenue"] / objective_performance["Transactions"],2))
objective_performance
```

Out[23]:

	objective	Customers	Campaigns	Transactions	Refunds	Signed_units_sold	Recorded_revenue	Average_transaction_value
2	Reactivation	21481	15	24173	723	28278	1955676.20	80.90
3	Retention	19884	13	22239	648	25980	1815008.12	81.61
1	Cross-sell	17831	12	19647	575	22903	1602569.22	81.57
0	Acquisition	14875	10	16113	469	18898	1310592.78	81.34

In [24]:
```
# How much recorded revenue is linked to campaigns targeting each segment?
target_segment_performance = (
    campaign_analysis
    .groupby("target_segment")
    .agg(
        Customers=("customer_id", "nunique"),
        Campaigns=("campaign_id", "nunique"),
        Transactions=("transaction_id", "nunique"),
        Refunds=("refund_flag","sum"),
        Signed_units_sold=("known_revenue_signed_units", "sum"),
        Recorded_revenue=("gross_revenue", "sum"))
    .reset_index()
    .sort_values("Recorded_revenue", ascending=False))
target_segment_performance["Average_transaction_value"] =(round(
    target_segment_performance["Recorded_revenue"] / target_segment_performance["Transactions"],2))
target_segment_performance
```

Out[24]:

	target_segment	Customers	Campaigns	Transactions	Refunds	Signed_units_sold	Recorded_revenue	Average_transaction_val
4	New Customers	18744	12	20745	578	24375	1697144.38	81.
2	Deal Seekers	15069	10	16325	499	19085	1321387.26	80.
1	Churn Risk	14947	11	16127	469	18945	1302849.85	80.
3	High Value	14008	9	15075	419	17620	1230434.85	81.
0	All	12944	8	13900	450	16034	1132029.98	81.

In [25]:
```
# Number of events
print("Total Events:",events["event_id"].nunique())
# Customers with digital activity
print("Active Customers:", events["customer_id"].nunique())
# Number of sessions
print("Total Sessions:", events["session_id"].nunique())
# Count how many times each event type occurs.
event_distribution = ( events["event_type"].value_counts().reset_index())
event_distribution.columns = ["Event_Type", "Total_Events"]
# Calculate each event type's percentage of all recorded event rows.
event_distribution["Percentage_%"] = (round(event_distribution["Total_Events"] /
```

```
                                                    event_distribution["Total_Events"].sum()* 100, 2))
event_distribution
```

```
Total Events: 2000000
Active Customers: 100000
Total Sessions: 633462
```

Out[25]:

	Event_Type	Total_Events	Percentage_%
0	view	1043573	52.18
1	click	379008	18.95
2	add_to_cart	284370	14.22
3	bounce	189922	9.5
4	purchase	103127	5.16

In [26]:

```
# Select completed, non-refunded transactions with a discount value.
discount_analysis = sales_analysis[
    (sales_analysis["refund_flag"] == 0) &
    (sales_analysis["gross_revenue"].notna()) &
    (sales_analysis["discount_applied"].notna())
].copy()
# How do transaction values differ across discount levels?
discount_stats = (
    discount_analysis
    .groupby("discount_applied")
    .agg(
        Transactions=("transaction_id", "nunique"),
        Average_Revenue=("gross_revenue", "mean"),
        Median_Revenue=("gross_revenue", "median"),
        Total_Revenue=("gross_revenue", "sum"))
    .reset_index())
# Convert 0.05 into 5%.
discount_stats["Discount_%"] = ( discount_stats["discount_applied"] * 100)
discount_stats
```

Out[26]:

	discount_applied	Transactions	Average_Revenue	Median_Revenue	Total_Revenue	Discount_%
0	0.00	54079	99.618451	73.340	5387266.22	0.0
1	0.05	13512	96.096624	70.955	1298457.58	5.0
2	0.10	10760	90.711397	66.210	976054.63	10.0
3	0.15	7255	85.722864	63.030	621919.38	15.0
4	0.20	4368	79.343292	58.920	346571.50	20.0

In [27]:

```
# Did recorded revenue increase or decrease from one month to the next?
sales_analysis["year_month"] = sales_analysis["timestamp"].dt.to_period("M")
monthly_performance = (
    sales_analysis
    .groupby("year_month")
    .agg(
        Customers=("customer_id", "nunique"),
        Transactions=("transaction_id", "nunique"),
        Refunds=("refund_flag","sum"),
        Signed_units_sold=("known_revenue_signed_units", "sum"),
        Recorded_revenue=("gross_revenue", "sum"))
    .reset_index()
    .sort_values("year_month"))
monthly_performance["MOM_Revenue_Change ($)"] = (
    monthly_performance["Recorded_revenue"] - monthly_performance["Recorded_revenue"].shift(1))
monthly_performance["MOM_Revenue_Growth (%)"] = (monthly_performance["Recorded_revenue"].pct_change(1) * 100).round(
monthly_performance.head()
```

Out[27]:

	year_month	Customers	Transactions	Refunds	Signed_units_sold	Recorded_revenue	MOM_Revenue_Change ($)	MOM_Revenue_
0	2021-01	2774	2813	59	3265	225536.72	NaN	
1	2021-02	2442	2473	76	2949	207915.83	-17620.89	
2	2021-03	2782	2812	78	3263	228099.36	20183.53	
3	2021-04	2675	2713	77	3191	225594.93	-2504.43	
4	2021-05	2821	2859	75	3425	237667.02	12072.09	

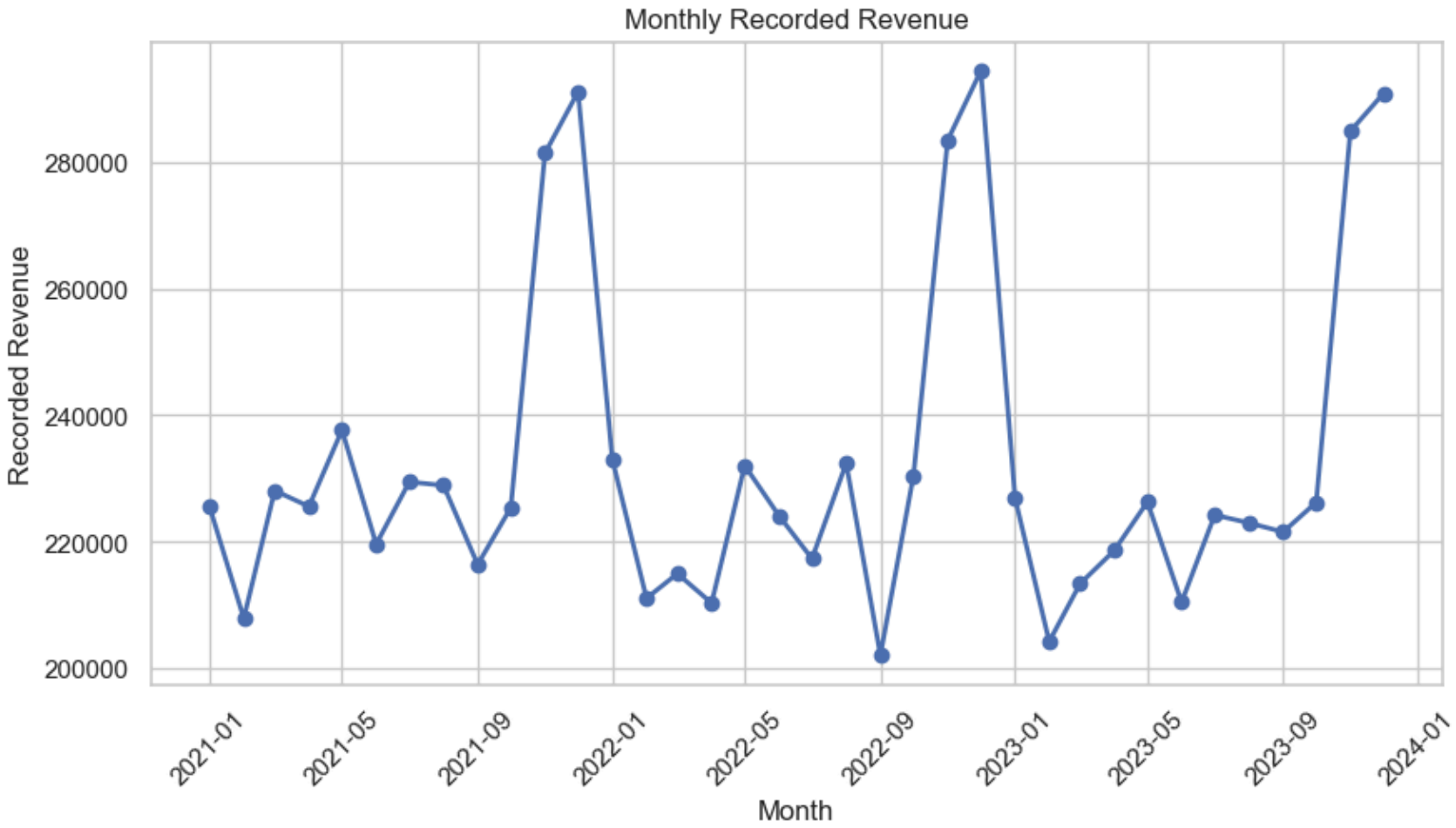
In [28]:

```
# How did annual recorded revenue change compared with the previous year?
sales_analysis["year"] = sales_analysis["timestamp"].dt.to_period("Y")
yearly_performance = (
    sales_analysis
    .groupby("year")
    .agg(
        Customers=("customer_id", "nunique"),
        Transactions=("transaction_id", "nunique"),
        Refunds=("refund_flag","sum"),
        Signed_units_sold=("known_revenue_signed_units", "sum"),
        Recorded_revenue=("gross_revenue", "sum"))
    .reset_index()
    .sort_values("year"))
yearly_performance["YoY_Revenue_Change ($)"] = (
    yearly_performance["Recorded_revenue"] - yearly_performance["Recorded_revenue"].shift(1))
yearly_performance["YoY_Revenue_Growth (%)"] = (yearly_performance["Recorded_revenue"].pct_change(1) * 100).round(2)
yearly_performance
```

Out[28]:

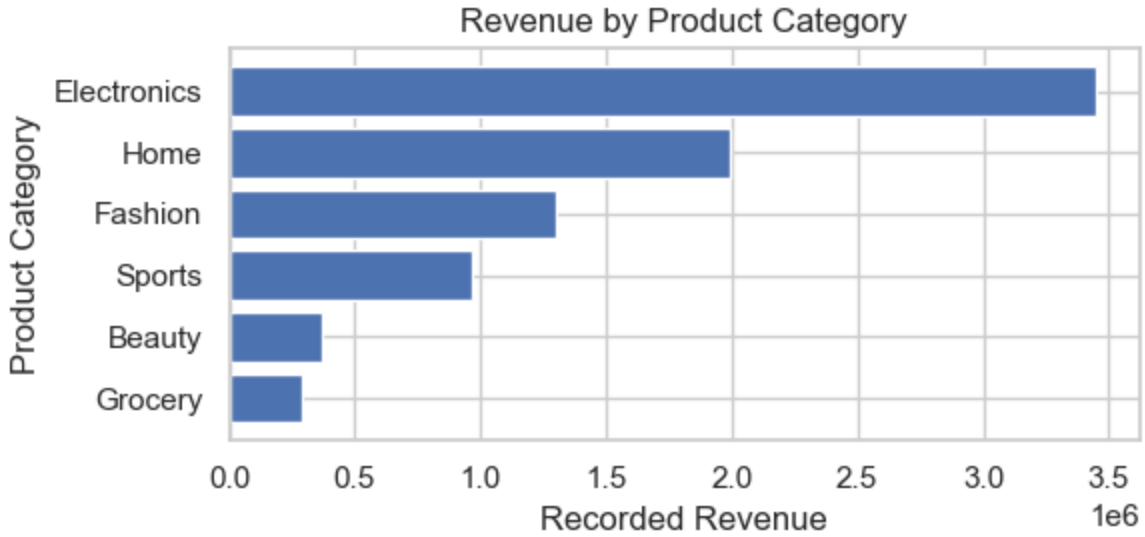
	year	Customers	Transactions	Refunds	Signed_units_sold	Recorded_revenue	YoY_Revenue_Change ($)	YoY_Revenue_Growth (%)
0	2021	29074	34474	959	40526	2817199.32	NaN	NaN
1	2022	28835	34187	1039	39998	2785927.47	-31271.85	-1.11
2	2023	29058	34466	1031	40004	2770839.57	-15087.90	-0.54

In [29]:
```
# Prepare a simple two-column table for the chart.
Monthly_revenue = (sales_analysis.groupby("year_month")["gross_revenue"].sum().reset_index(name="Revenue"))
# Convert the month column into dates.
Monthly_revenue["year_month"] = (Monthly_revenue["year_month"].dt.to_timestamp())
# Sort the months in chronological order.
Monthly_revenue = Monthly_revenue.sort_values("year_month")
# Create a folder called graphs.
FIG = Path("graphs")
FIG.mkdir(exist_ok=True)
# Create the figure
plt.figure(figsize=(10, 5))
# Create a line chart
plt.plot(Monthly_revenue["year_month"], Monthly_revenue["Revenue"],
     marker="o",
    linewidth=2)
# Add title and labels
plt.title("Monthly Recorded Revenue")
plt.xlabel("Month")
plt.ylabel("Recorded Revenue")
# Improve spacing
plt.xticks(rotation=45)
# Save the graph inside the graphs folder.
plt.savefig(FIG / "01_monthly_revenue.png", dpi=160)
# Display the graph
plt.show()
```

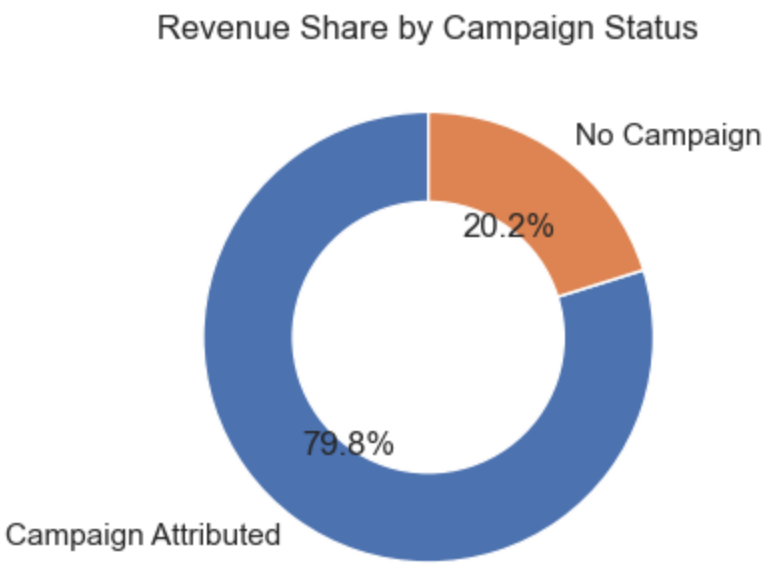


In [30]:
```
# Sort categories from lowest to highest revenue
category_plot = category_performance.sort_values("Recorded_revenue")
# Create the figure
plt.figure(figsize=(6, 3))
# Create a horizontal bar chart
plt.barh(
    category_plot["category"],
    category_plot["Recorded_revenue"])
# Add title and labels
plt.title("Revenue by Product Category")
plt.xlabel("Recorded Revenue")
plt.ylabel("Product Category")
```

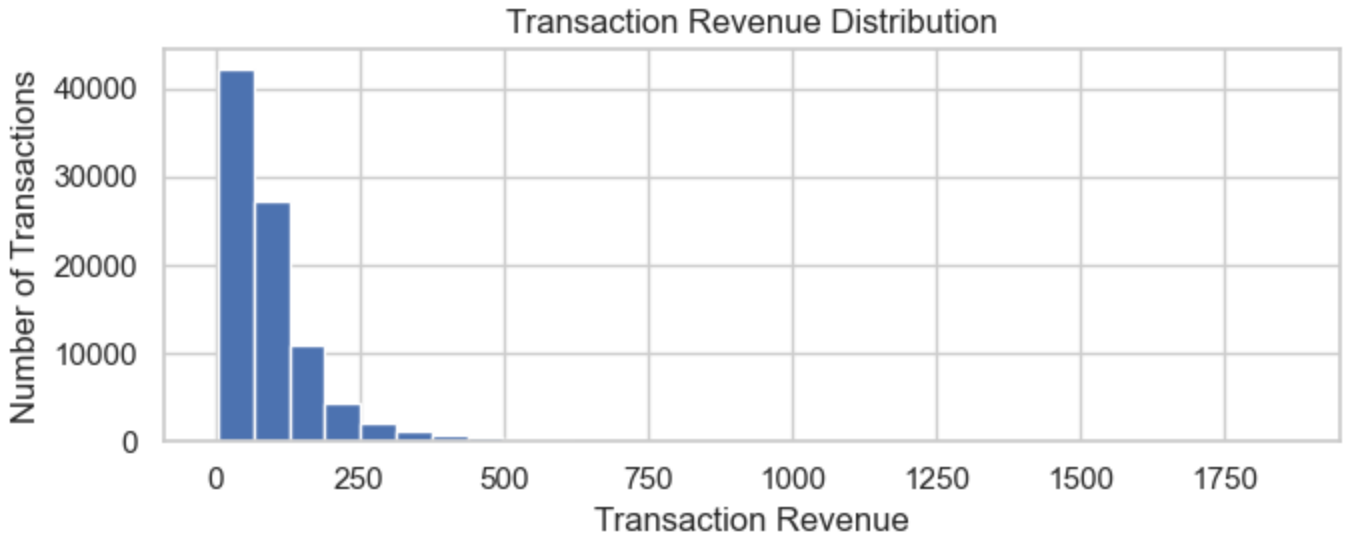
```
# Improve spacing
plt.tight_layout()
# Save the graph inside the graphs folder.
plt.savefig(FIG / "02_category_revenue.png", dpi=160)
# Display the graph
plt.show()
```



In [63]:
```
# Create the figure
plt.figure(figsize=(4, 4))
# Create a donut chart
plt.pie(
    campaign_status_performance["Recorded_revenue"],
    labels=campaign_status_performance["campaign_status"],
    autopct="%1.1f%%",
    startangle=90,
    wedgeprops={"width": 0.4})
# Add title
plt.title("Revenue Share by Campaign Status")
# Improve spacing
plt.tight_layout()
# Save the graph inside the graphs folder.
plt.savefig(FIG / "03_campaign_donut.png", dpi=160)
# Display the graph
plt.show()
```

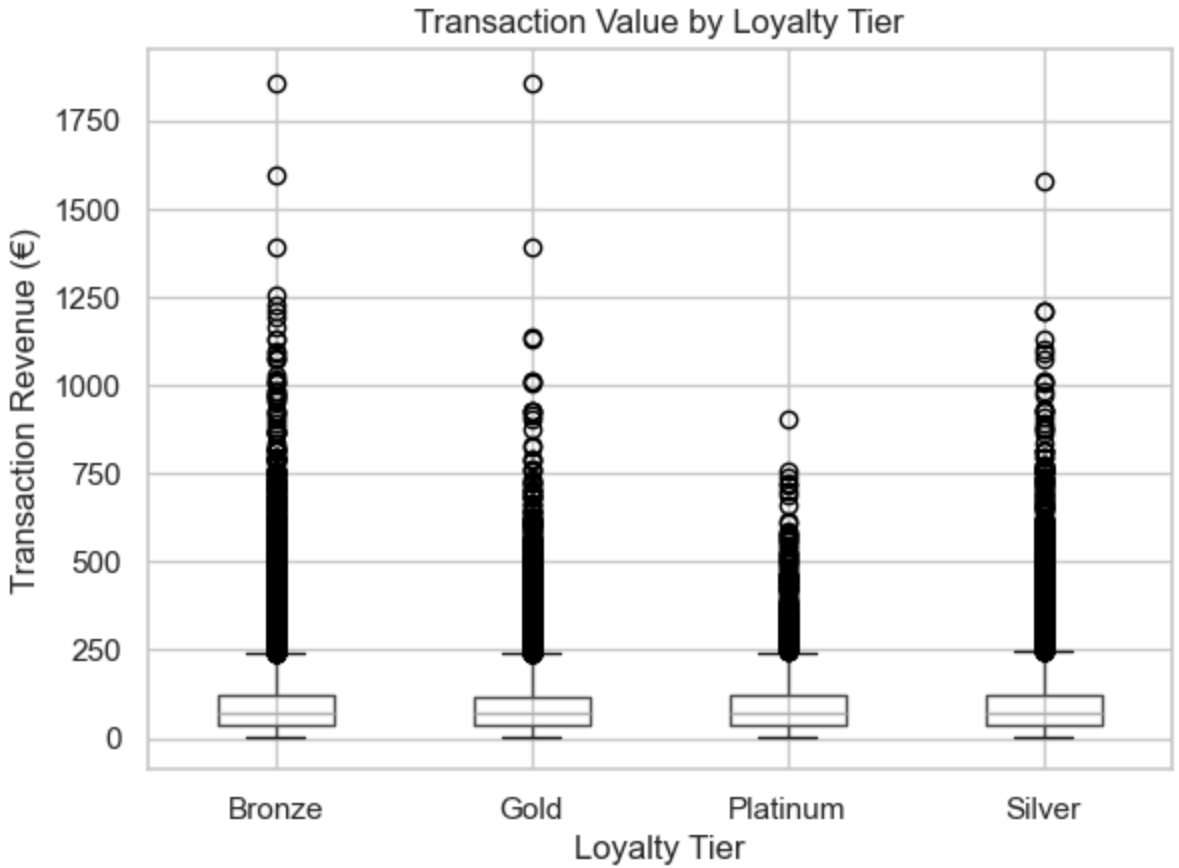


In [31]:
```
# Create the figure
plt.figure(figsize=(7, 3))
# Create a histogram
plt.hist(
    completed_sales["gross_revenue"],
    bins=30)
# Add title and labels
plt.title("Transaction Revenue Distribution")
plt.xlabel("Transaction Revenue")
plt.ylabel("Number of Transactions")
# Improve spacing
plt.tight_layout()
# Save the graph inside the graphs folder.
plt.savefig(FIG / "04_revenue_histogram.png", dpi=160)
# Display the graph
plt.show()
```



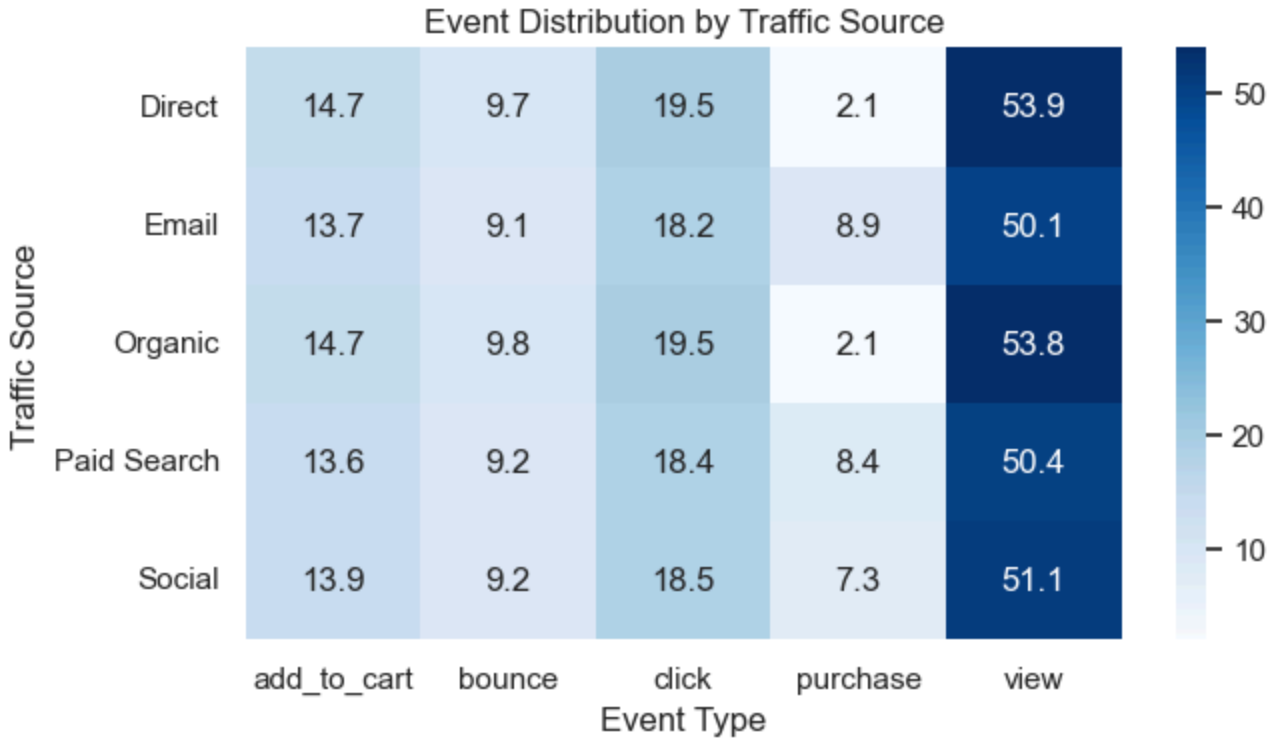
In [32]:
```
stat_sales = customer_analysis[(customer_analysis["refund_flag"] == 0) &(customer_analysis["gross_revenue"].notna())]
plt.figure(figsize=(7, 4))
stat_sales.boxplot(
    column="gross_revenue",
    by="loyalty_tier")
plt.title("Transaction Value by Loyalty Tier")
plt.suptitle("")
plt.xlabel("Loyalty Tier")
plt.ylabel("Transaction Revenue (€)")
# Save the graph inside the graphs folder.
plt.savefig(FIG / "05_loyalty_boxplot.png", dpi=160)
plt.show()
```

```
<Figure size 700x400 with 0 Axes>
```



In [66]:
```
# Calculate the percentage of each event type within each traffic source
event_heat = pd.crosstab(events["traffic_source"],events["event_type"],normalize="index") * 100
# Display the table
event_heat
# Create the figure
plt.figure(figsize=(7, 4))
# Create a heatmap
sns.heatmap(
    event_heat,
    annot=True,
    fmt=".1f",
    cmap="Blues")
# Add title and labels
plt.title("Event Distribution by Traffic Source")
```

```
plt.xlabel("Event Type")
plt.ylabel("Traffic Source")
# Improve spacing
plt.tight_layout()
# Save the graph inside the graphs folder.
plt.savefig(FIG / "06_traffic_heatmap.png", dpi=160)
# Display the graph
plt.show()
```



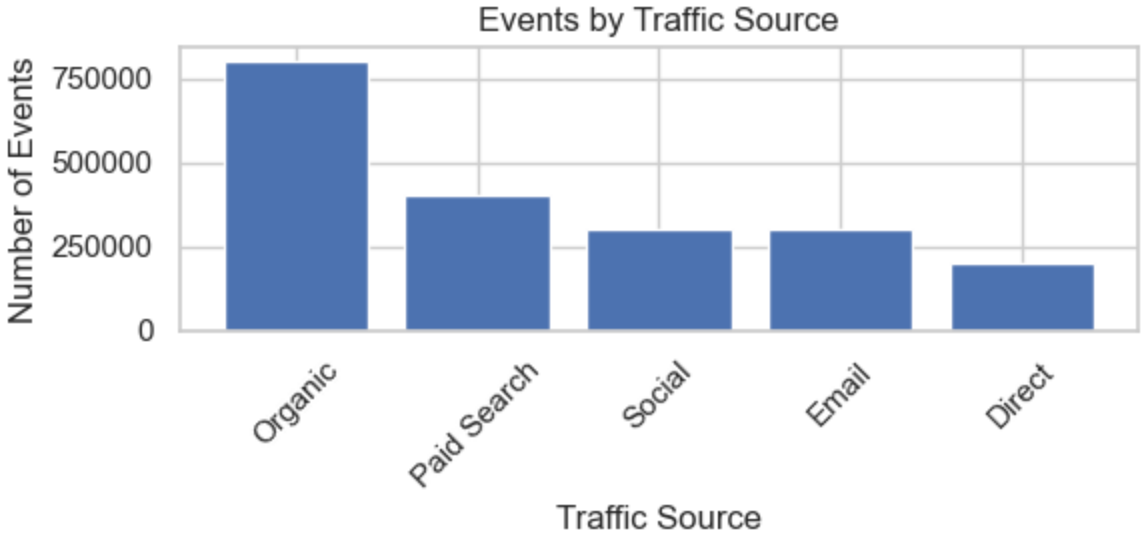
In [33]:
```
# Sort categories by refund rate
refund_plot = category_performance.sort_values("Refund_rate", ascending=False)
# Create the figure
plt.figure(figsize=(6, 3))
# Create a bar chart
plt.bar(
    refund_plot["category"],
    refund_plot["Refund_rate"])
# Add title and labels
plt.title("Refund Rate by Product Category")
```

```
plt.xlabel("Product Category")
plt.ylabel("Refund Rate (%)")
# Rotate the category names
plt.xticks(rotation=45)
# Improve spacing
plt.tight_layout()
# Save the graph inside the graphs folder.
plt.savefig(FIG / "08_category_refunds.png", dpi=160)
# Display the graph
plt.show()
```



In [34]:
```
# Count events for each traffic source
source_counts = events["traffic_source"].value_counts()
# Display the result
source_counts
# Create the figure
plt.figure(figsize=(6, 3))
# Create a bar chart
plt.bar(
    source_counts.index,
    source_counts.values)
# Add title and labels
plt.title("Events by Traffic Source")
plt.xlabel("Traffic Source")
```

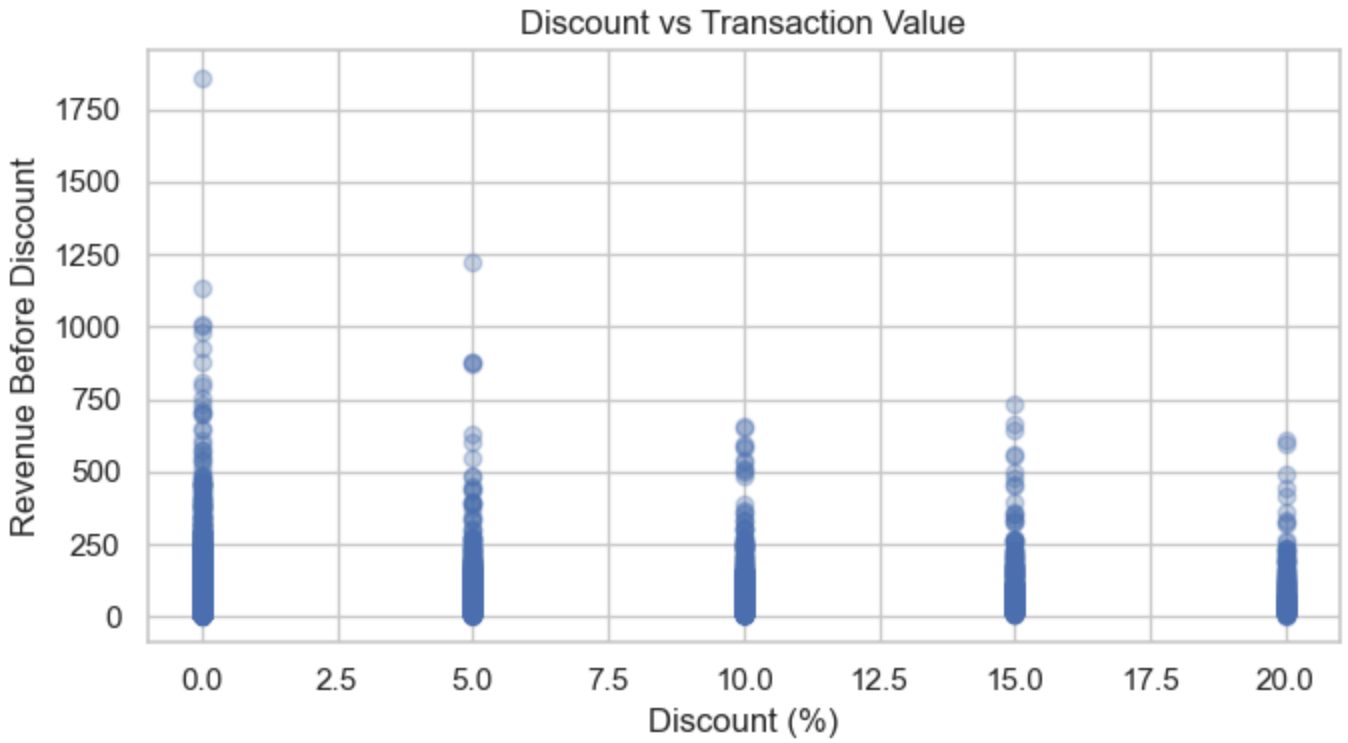
```
plt.ylabel("Number of Events")
# Rotate traffic-source names
plt.xticks(rotation=45)
# Improve spacing
plt.tight_layout()
# Save the graph inside the graphs folder.
plt.savefig(FIG / "09_source_event_counts.png", dpi=160)
# Display the graph
plt.show()
```



In [35]:
```
# Select non-refunded transactions with known values
scatter_data = sales_analysis[
    (sales_analysis["refund_flag"] == 0) &
    (sales_analysis["revenue_before_discount"].notna())]
# Select 5% of transactions to avoid too many dots
scatter_plot = scatter_data.sample(frac=0.05, random_state=42)
# Create the figure
plt.figure(figsize=(7, 4))

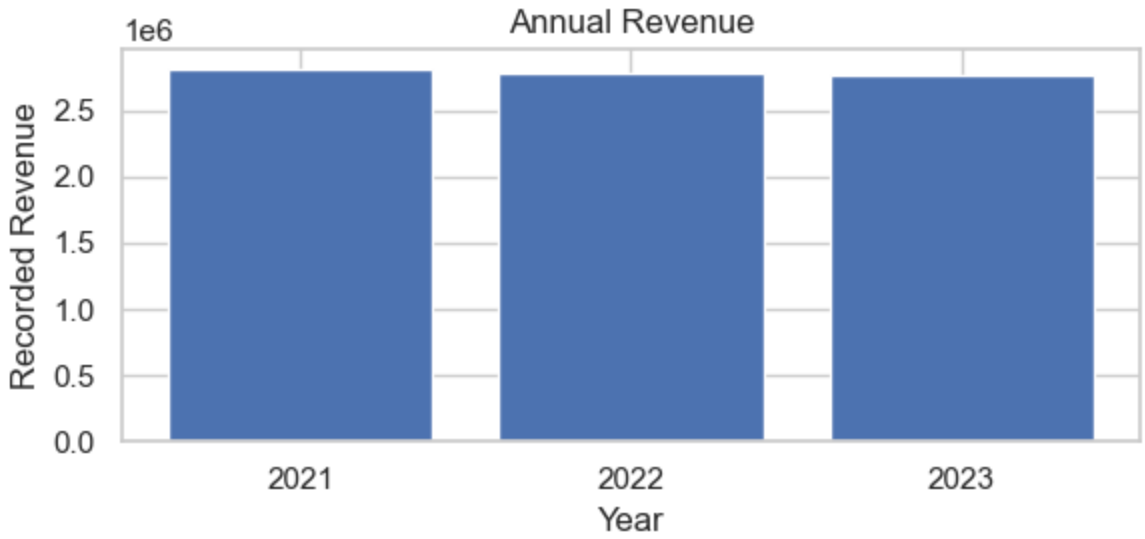
# Create a scatter plot
plt.scatter(
    scatter_plot["discount_applied"] * 100,
    scatter_plot["revenue_before_discount"],
    alpha=0.3)
```

```
# Add title and labels
plt.title("Discount vs Transaction Value")
plt.xlabel("Discount (%)")
plt.ylabel("Revenue Before Discount")
# Improve spacing
plt.tight_layout()
# Save the graph inside the graphs folder.
plt.savefig(FIG / "10_Discount vs Transaction Value", dpi=160)
# Display the graph
plt.show()
```



In [36]:
```
Annual_revenue = (sales_analysis
    .groupby("year")["gross_revenue"]
    .sum()
    .reset_index(name="Revenue"))
# Sort years from oldest to newest
annual_plot = Annual_revenue.sort_values("year")
# Create the figure
```

```
plt.figure(figsize=(6, 3))
# Create a bar chart
plt.bar(
    annual_plot["year"].astype(str),
    annual_plot["Revenue"])
# Add title and labels
plt.title("Annual Revenue")
plt.xlabel("Year")
plt.ylabel("Recorded Revenue")
# Improve spacing
plt.tight_layout()
# Save the graph inside the graphs folder.
plt.savefig(FIG / "11_Annual Revenue", dpi=160)
# Display the graph
plt.show()
```



In [ ]: